HW2 Report: Multi-VM Distributed System with RPC, File Service, and Pub/Sub

Amir Hosein Sabahi - 810101556

Amir Mahdi Zaryoun - 810100157

Environment

The implementation and execution were done on three Ubuntu virtual machines created and managed with VMware Workstation. Each service was executed on a separate VM and all service-to-service communication used network IP addresses, not localhost.

Environment Summary

Item	Value
Virtualization platform	VMware Workstation
Guest operating system	Ubuntu Linux VMs
Implementation language	Go
Go dependencies	Only standard-library packages
RPC technology	JSON-RPC over HTTP between VM1 and VM2
File service	HTTP File Server implemented with Go on VM3
Pub/Sub technology	Simple HTTP-based broker and subscriber implemented in Go
Password storage	SHA-256 password hashes stored in users.json on VM2
Network adapter	VMware adapter enabled with Connected and Connect at power on
IP assignment	DHCP; IPs may change after network or VMware mode changes
Current execution IPs	VM1 web-vm = 192.168.29.130; VM2 auth-vm = 192.168.29.131; VM3 file-vm = 192.168.29.129

Table of Contents

1. Overview

This homework implements a small distributed system using multiple virtual machines. The design separates the web interface, authentication logic, user data, file storage, and system monitoring logic into different services.

The browser communicates with VM1. VM1 displays the login page and forwards authentication requests to VM2 using JSON-RPC. VM2 stores the user database and returns the login result. After successful login, VM1 displays an image loaded through the network from VM3. In the third part, VM1 also monitors its memory usage and publishes HIGH_MEMORY_USAGE events to a simple Pub/Sub broker when memory crosses the threshold.

- VM1: web-vm - web server, login UI, JSON-RPC client, memory publisher
- VM2: auth-vm - authentication server and users.json storage
- VM3: file-vm - HTTP file and image server
- pubsub: publisher.go runs the simple broker, and subscriber.go receives alerts

2. Final Project Structure

The final folder is named HW2. It follows the project structure requested in the homework statement and contains independent folders for the web service, authentication service, file service, Pub/Sub components, and README files.

```
Project Structure

 HW2/
 web-vm/
   main.go
   templates/
     login.html
     welcome.html
   static/
     styles.css
   README.md
 auth-vm/
   main.go
   users.json
   README.md
 file-vm/
   main.go
   files/
     info.txt
     c.txt
   images/
     sample.svg
     Saul.jpg
     messi.jpg
   README.md
 pubsub/
   publisher.go
   subscriber.go
   README.md
 README.md
```

Figure 1. Project structure used for the final submission.

Folder	Purpose
web-vm	Go web server, HTML templates, CSS, login flow, memory endpoints
auth-vm	JSON-RPC authentication service and users.json with SHA-256 password hashes
file-vm	HTTP file server with files/ and images/ directories
pubsub	HTTP-based Pub/Sub broker in publisher.go and subscriber client in subscriber.go

```
oteook@Amirhosein:/mnt/f/Term8/Dist/HW2$ scp -r ./web-vm amirhosein@192.168.29.130:~/
scp -r ./pubsub amirhosein@192.168.29.130:~/
scp -r ./auth-vm amirhosein@192.168.29.131:~/
scp -r ./file-vm amirhosein@192.168.29.129:~/
amirhosein@192.168.29.130's password:
main.go                                       100% 3576     1.7MB/s   00:00
README.md                                     100%  309   144.1KB/s   00:00
styles.css                                    100% 1173   620.7KB/s   00:00
login.html                                    100%  717   483.3KB/s   00:00
welcome.html                                  100%  553   293.9KB/s   00:00
amirhosein@192.168.29.130's password:
publisher.go                                  100% 2993     1.5MB/s   00:00
README.md                                     100%  214   149.8KB/s   00:00
subscriber.go                                 100% 1463   992.9KB/s   00:00
amirhosein@192.168.29.131's password:
main.go                                       100% 3080     1.0MB/s   00:00
README.md                                     100%  300   159.4KB/s   00:00
users.json                                    100%  367   145.0KB/s   00:00
amirhosein@192.168.29.129's password:
c.txt                                         100%  272    99.6KB/s   00:00
info.txt                                      100%   82    47.7KB/s   00:00
messi.jpg                                     100%  218KB  22.0MB/s   00:00
sample.svg                                    100%  793   257.7KB/s   00:00
Saul.jpg                                      100%   82KB  20.1MB/s   00:00
main.go                                       100%  683   373.9KB/s   00:00
README.md                                     100%  206   120.3KB/s   00:00
```

Figure 2. Files were copied from the host/WSL environment to the correct VMs using scp.

3. VMware Virtual Machine Setup

VMware Workstation was used to create three separate Ubuntu virtual machines. The network adapter of each VM was enabled and kept on the same reachable network so that VM1, VM2, and VM3 could communicate using their IP addresses.

VM	Hostname	Role	IP	Ports
VM1	web-vm	Web server and Pub/Sub components	192.168.29.130	8080, 7000
VM2	auth-vm	Authentication service and user data	192.168.29.131	9000
VM3	file-vm	File and image service	192.168.29.129	9090

3.1. VMware Network Notes

- Each VM used a VMware virtual network adapter.
- The adapter options Connected and Connect at power on were enabled.
- The VMs received IPv4 addresses through DHCP.
- The IP addresses changed during testing when the network mode or network source changed.
- Before each run, hostname -I or ip -br a was used to confirm the current VM IPs.

This IP-change behavior is important for reproducibility. The service code is not hard-coded to one IP. Instead, VM1 receives AUTH_ADDR, FILE_URL, BROKER_URL, and MEMORY_THRESHOLD_MB as environment variables when it is started.

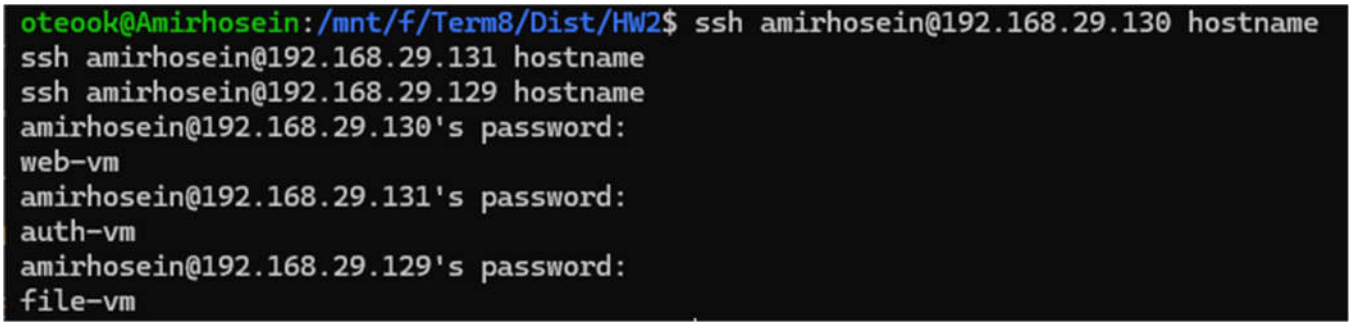

Figure 3. SSH hostname checks confirm the mapping of IP addresses to web-vm, auth-vm, and file-vm.

4. System Architecture

The system contains three main service VMs. VM1 receives browser requests. VM1 does not store user data and does not authenticate users locally. Instead, it calls VM2 using JSON-RPC. VM3 acts as an independent file and image repository. Pub/Sub monitoring is added on VM1 with a simple HTTP broker and subscriber.

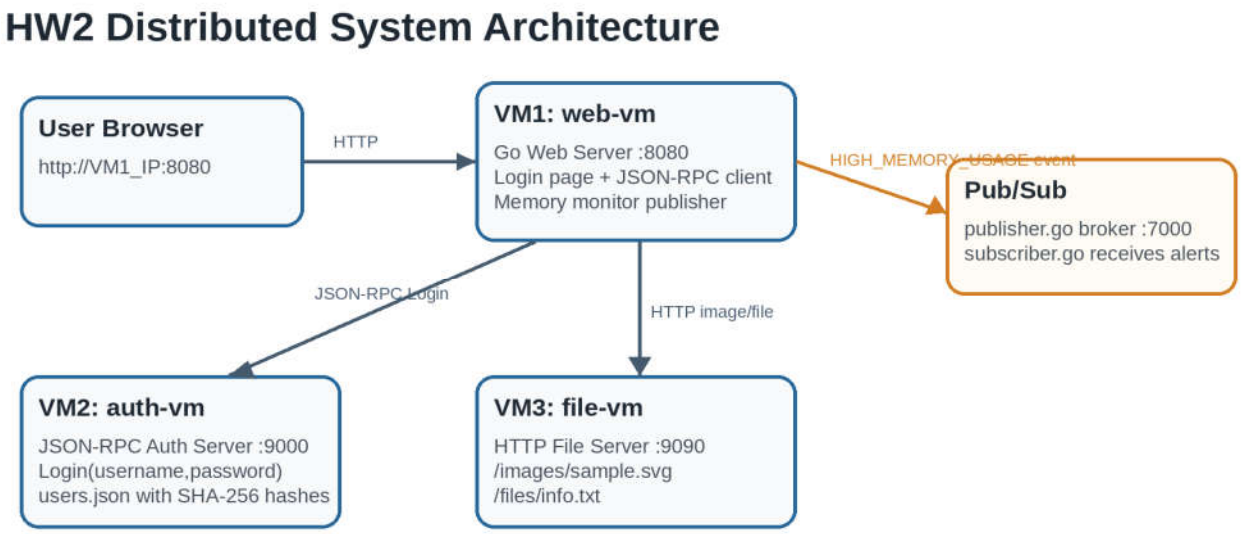

Figure 4. Overall architecture of the distributed system.

Communication	Protocol	Reason
Browser -> VM1	HTTP	User opens login page and submits the form
VM1 -> VM2	JSON-RPC over HTTP	Remote Login procedure call
VM1 -> VM3	HTTP file access	Image/file loaded by network URL
VM1 -> Pub/Sub broker	HTTP POST	Publish memory event
Subscriber -> Pub/Sub broker	HTTP streaming style response	Receive events and print alert

5. RPC Authentication Design

JSON-RPC was selected for communication between VM1 and VM2. It is simple, readable during testing, and still represents a real remote procedure call model. The remote procedure implemented by VM2 is Login(username, password).

Item	Value
RPC technology	JSON-RPC 2.0
Transport	HTTP POST
Endpoint	http://VM2_IP:9000/rpc
Method	Login
Inputs	username, password
Output	success flag and message
Direct user data access from VM1	No

5.1. Example RPC Request

```
curl -X POST http://192.168.29.131:9000/rpc -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","method":"Login","params":{"username":"alice","password":"alice123"},"id":1}'
```

5.2. Example Successful Response

```
{"jsonrpc":"2.0","result":{"success":true,"message":"login_successful"},"id":1}
```

6. VM2 Authentication Service

VM2 runs the authentication service on port 9000. The service reads users from users.json. In the final version, passwords are not stored as raw text. Each stored password is represented as a SHA-256 hash. When a login request arrives, VM2 hashes the received password and compares it with the stored password_hash value.

Endpoint	Purpose
GET /health	Checks whether the auth service is running
POST /rpc	Accepts JSON-RPC Login requests

- users.json is stored only on VM2.
- VM1 has no direct access to users.json.
- At least three test users are defined: alice, bob, and admin.
- Password hashes improve the design compared with storing plain passwords.

Figure 5. Auth service tests: health check, successful Login RPC, and failed Login RPC.

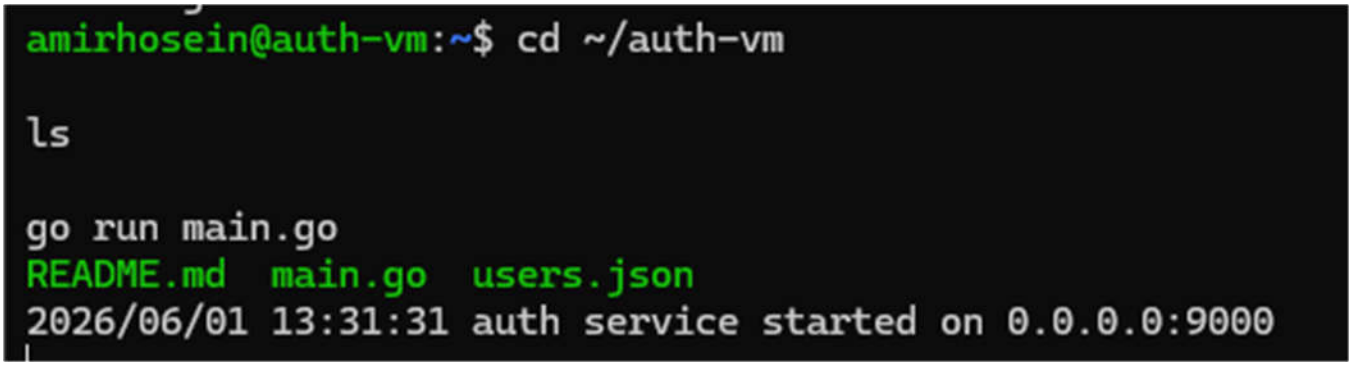

Figure 6. Authentication service running on auth-vm at port 9000.

7. VM3 File and Image Service

VM3 acts as the file and image repository. A Go HTTP file server listens on port 9090 and serves files from two folders: files/ and images/. VM1 uses a network URL to display the image after successful login. The file is not copied into VM1 and is not loaded through a local path.

Path	Purpose
/health	File service health check
/files/info.txt	Text file evidence served from VM3
/images/messi.jpg	Image displayed by VM1 after login
/images/sample.svg	Additional sample image available from VM3

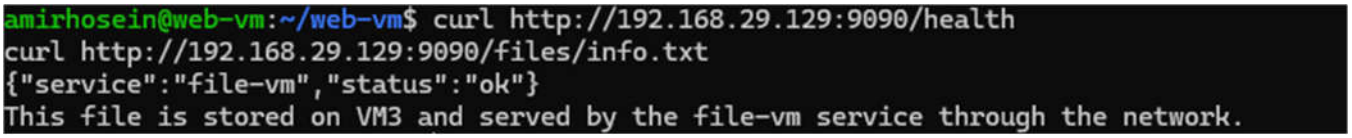

Figure 7. VM1 can access VM3 health endpoint and info.txt through the network

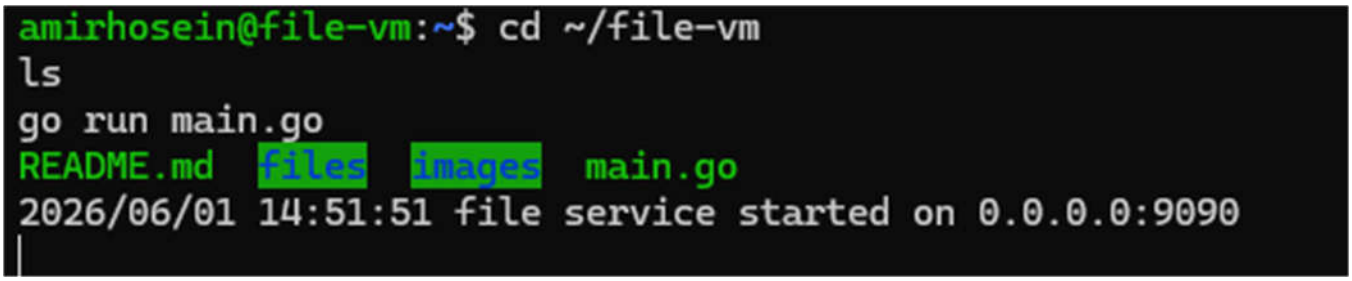

Figure 8. File service running on file-vm at port 9090.

8. VM1 Web Server and Login Flow

VM1 runs the web service on port 8080. It serves the login page, accepts username and password through an HTML form, and calls VM2 with JSON-RPC. VM1 does not check the password locally. On successful authentication, the welcome page is rendered and the image URL points to VM3.

Endpoint	Purpose
GET / or /login	Display login form
POST /login	Read form fields and call VM2 using JSON-RPC
GET /health	Web service health check
GET /memory	Show current memory usage for Part 3
GET /consume-memory?mb=N	Allocate memory intentionally for testing alerts

8.1. Web Server Run Command

```
AUTH_ADDR=192.168.29.131:9000
FILE_URL=http://192.168.29.129:9090/images/messi.jpg go run main.go
```

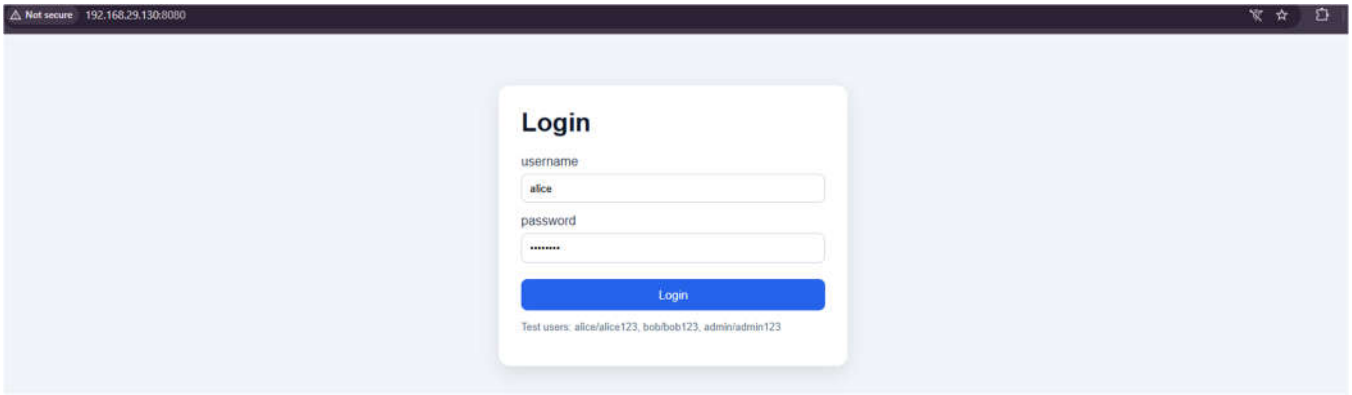

Figure 9. Browser login page served by VM1 at http://192.168.29.130:8080.

Figure 10. Successful login and image loaded from VM3 through the network.

8.2. Part 2 Test Scenario Evidence

1. The user opened the VM1 web page from the host browser.
2. The login form asked for username and password.
3. Wrong credentials returned an error result.
4. Correct credentials were sent from VM1 to VM2 through JSON-RPC.
5. VM2 returned success for alice/alice123.
6. VM1 displayed the welcome page.
7. The welcome page loaded messi.jpg from VM3 using the VM3 HTTP URL.

The evidence screenshots show that the system uses three distinct IP addresses. This confirms that the services are not all running on a single localhost-only setup.

9. Pub/Sub and Memory Monitoring

For Part 3, a simple HTTP-based Pub/Sub mechanism was implemented in Go. The web service on VM1 acts as the publisher of memory events. The file pubsub/publisher.go runs a small broker on port 7000. The subscriber program connects to the broker and waits for events. When the web service memory usage crosses the threshold, the broker forwards the event to the subscriber and the subscriber prints an alert.

Component	Role
web-vm/main.go	Periodically checks memory and publishes HIGH_MEMORY_USAGE events
pubsub/publisher.go	Runs the simple HTTP broker on port 7000
pubsub/subscriber.go	Subscribes to events and prints alerts
MEMORY_THRESHOLD_MB	Defines the threshold; 300 MB was used
/consume-memory?mb=N	Allocates memory intentionally and keeps it in memory

9.1. Memory Metrics

The implementation uses Go runtime memory information. The memory endpoint reports values such as alloc_mb, allocated_mb, heap_alloc_mb, sys_mb, and threshold_mb. For example, allocated_mb indicates the intentionally retained memory allocated through /consume-memory.


```
{"alloc_mb":100,"allocated_mb":100,"heap_alloc_mb":100,"sys_mb":107,"threshold_mb
":300}
```

9.2. Pub/Sub Execution Evidence

The broker was started on VM1. After the subscriber connected, the web service published a HIGH_MEMORY_USAGE event with memory=350 and threshold=300.

```
amirhosein@web-vm:~$ cd ~/pubsub
ls
go run publisher.go
README.md  publisher.go  subscriber.go
2026/06/01 14:33:37 PubSub broker started on 0.0.0.0:7000
2026/06/01 14:34:10 subscriber connected
2026/06/01 14:36:30 published event=HIGH_MEMORY_USAGE memory=350 threshold=300 subscribers=1
```

Figure 11. Pub/Sub broker started and later published the HIGH_MEMORY_USAGE event.

The subscriber connected to the broker and printed a clear alert after receiving the memory event.

```
amirhosein@web-vm:~$ cd ~/pubsub
ls
SUBSCRIBE_URL=http://192.168.29.130:7000/events go run subscriber.go
README.md  publisher.go  subscriber.go
2026/06/01 14:34:10 subscriber connected to http://192.168.29.130:7000/events
ALERT: HIGH MEMORY USAGE
service: web-server
memory_mb: 350
threshold_mb: 300
timestamp: 2026-06-01T14:36:30Z
event_type: HIGH_MEMORY_USAGE
```

Figure 12. Subscriber receives the event and prints ALERT: HIGH MEMORY USAGE.

9.3. Web Service Memory Event Evidence

The VM1 web service was executed with AUTH_ADDR, FILE_URL, BROKER_URL, and MEMORY_THRESHOLD_MB. After the consume-memory endpoint increased the allocation past 300 MB, the web service logged that the memory event was published.

Figure 13. Web service started with broker configuration and published a 350 MB memory event.

9.4. Expected Event Format

```
{
  "event_type": "HIGH_MEMORY_USAGE",
  "service": "web-server",
  "memory_mb": 350,
  "threshold_mb": 300,
  "timestamp": "2026-06-01T14:36:30Z"
}
```

10. Build and Run Commands

The following commands reproduce the final run. The exact IP addresses must be updated if DHCP assigns different addresses.

10.1. Verify VM Addresses

```
ssh amirhosein@192.168.29.130 hostname
ssh amirhosein@192.168.29.131 hostname
ssh amirhosein@192.168.29.129 hostname
```

10.2. Copy Files to VMs

```
scp -r ./web-vm amirhosein@192.168.29.130:~/
scp -r ./pubsub amirhosein@192.168.29.130:~/
scp -r ./auth-vm amirhosein@192.168.29.131:~/
scp -r ./file-vm amirhosein@192.168.29.129:~/
```

10.3. Run VM2 Auth Service

```
ssh amirhosein@192.168.29.131
cd ~/auth-vm
go run main.go
```

10.4. Run VM3 File Service

```
ssh amirhosein@192.168.29.129
cd ~/file-vm
go run main.go
```

10.5. Run Pub/Sub Broker and Subscriber on VM1

```
ssh amirhosein@192.168.29.130
cd ~/pubsub
go run publisher.go
```

```
ssh amirhosein@192.168.29.130
cd ~/pubsub
SUBSCRIBE_URL=http://192.168.29.130:7000/events go run subscriber.go
```

10.6. Run VM1 Web Service

```
ssh amirhosein@192.168.29.130
cd ~/web-vm
AUTH_ADDR=192.168.29.131:9000
FILE_URL=http://192.168.29.129:9090/images/messi.jpg
BROKER_URL=http://192.168.29.130:7000/publish MEMORY_THRESHOLD_MB=300 go run
main.go
```

10.7. Test Commands

```
curl http://192.168.29.131:9000/health
curl http://192.168.29.129:9090/health
curl http://192.168.29.130:8080/health
curl http://192.168.29.130:8080/memory
curl "http://192.168.29.130:8080/consume-memory?mb=100"
```

11. Challenges and Solutions

11.1. VM IP Address Changes

One challenge was that VM IP addresses changed when VMware network settings or the host network changed. This was caused by DHCP. The solution was to check hostname -I or ip -br a before every run and pass the current IPs through environment variables instead of hard-coding them into the code.

11.2. Separating Authentication from the Web Server

The web server only sends a JSON-RPC Login request to VM2. It does not open users.json and does not compare passwords locally. This keeps user data isolated on VM2.

11.3. Proving VM3 File Retrieval

The file URL displayed in the welcome page points to the VM3 address, such as http://192.168.29.129:9090/images/messi.jpg. This proves that the image is loaded through the network from the file VM.

11.4. Memory Alert Testing

To make the memory alert test reproducible, the endpoint /consume-memory?mb=N was implemented. Allocated memory is retained in a global structure so it is not immediately freed by Go garbage collection.

12. Conclusion and Submission Notes

The final project demonstrates a distributed system deployed across multiple VMware Ubuntu virtual machines. VM1 provides the user-facing web interface, VM2 performs authentication through JSON-RPC and stores user data, and VM3 provides file and image resources through HTTP. The system also includes a simple Pub/Sub mechanism for memory alerts.

The implementation satisfies the main distributed computing requirements: separation of services, network-based communication, RPC authentication, independent file service, memory monitoring, event publishing, subscriber alert handling, and reproducible run commands.

Final Submission Notes

- Submit the complete HW2 folder with web-vm, auth-vm, file-vm, pubsub, screenshots, README files, and this report.
- Before running on a new system, check VM IP addresses again because DHCP may assign new IPs.
- Update AUTH_ADDR, FILE_URL, BROKER_URL, and SUBSCRIBE_URL based on the new IPs.
- Run VM2 and VM3 first, then Pub/Sub broker/subscriber, then VM1 web service.